

RMA: Rapid Motor Adaptation for Legged Robots

Ashish Kumar
UC Berkeley

Zipeng Fu
Carnegie Mellon University

Deepak Pathak
Carnegie Mellon University

Jitendra Malik
UC Berkeley, Facebook

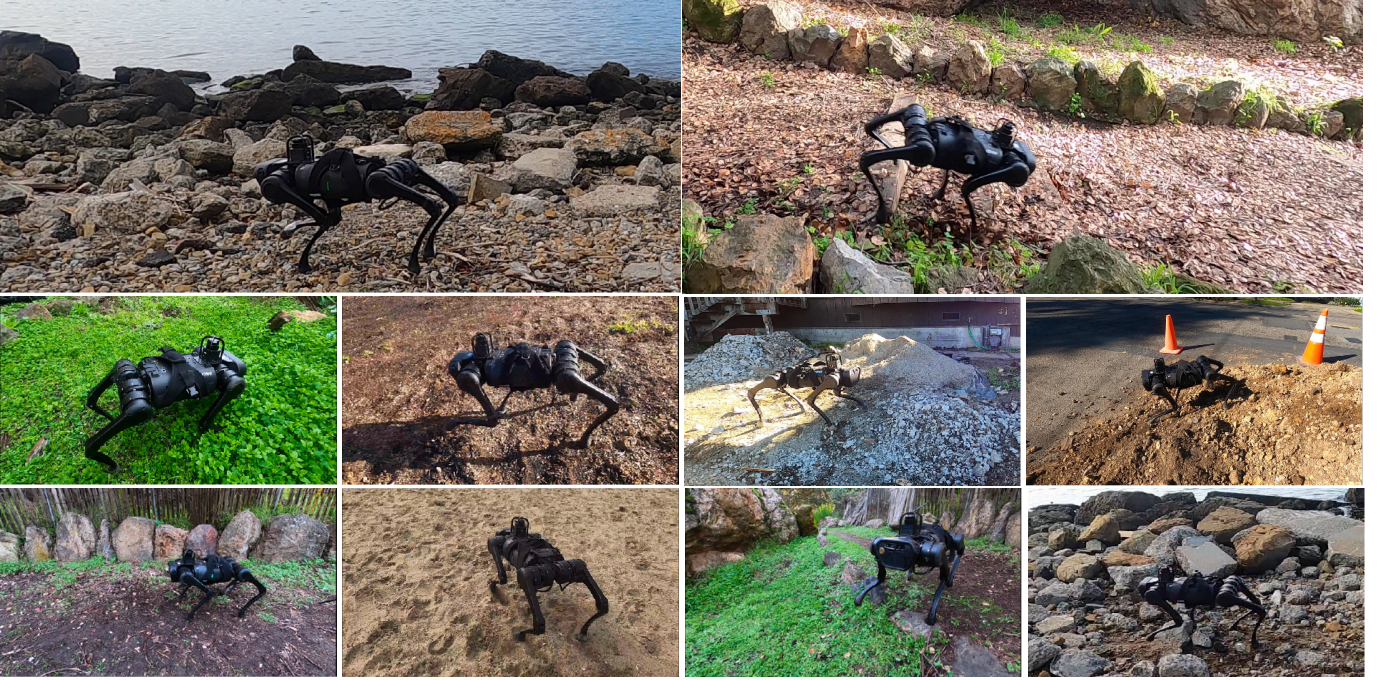


Fig. 1: We demonstrate the performance of RMA on several challenging environments. The robot is successfully able to walk on sand, mud, hiking trails, tall grass and dirt pile without a single failure in all our trials. The robot was successful in 70% of the trials when walking down stairs along a hiking trail, and succeeded in 80% of the trials when walking across a cement pile and a pile of pebbles. The robot achieves this high success rate despite never having seen unstable or sinking ground, obstructive vegetation or stairs during training. All deployment results are with the same policy without any simulation calibration, or real-world fine-tuning. Videos at <https://ashish-kmr.github.io/rma-legged-robots/>

Abstract—Successful real-world deployment of legged robots would require them to *adapt in real-time* to unseen scenarios like changing terrains, changing payloads, wear and tear. This paper presents Rapid Motor Adaptation (RMA) algorithm to solve this problem of real-time online adaptation in quadruped robots. RMA consists of two components: **a base policy and an adaptation module**. The combination of these components enables the robot to adapt to novel situations in fractions of a second. RMA is trained completely in simulation without using any domain knowledge like reference trajectories or predefined foot trajectory generators and is deployed on the A1 robot without any fine-tuning. We train RMA on a varied terrain generator using bioenergetics-inspired rewards and deploy it on a variety of difficult terrains including rocky, slippery, deformable surfaces in environments with grass, long vegetation, concrete, pebbles, stairs, sand, etc. RMA shows state-of-the-art performance across diverse real-world as well as simulation experiments. Video results at <https://ashish-kmr.github.io/rma-legged-robots/>.

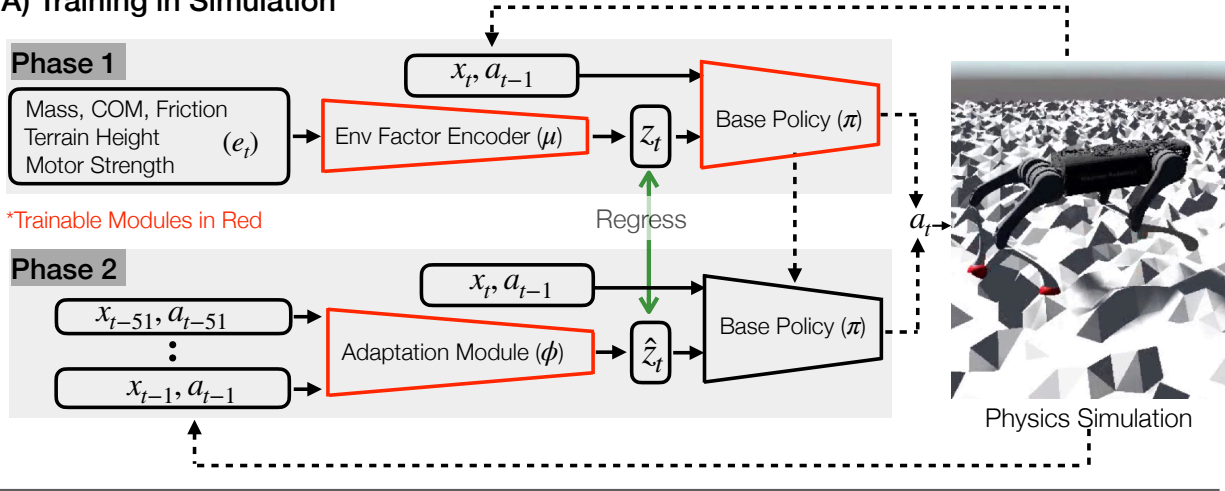
I. INTRODUCTION

Great progress has been made in legged robotics over the last forty years through the modeling of physical dynamics and the

tools of control theory [36, 43, 46, 16, 56, 63, 50, 26, 28, 2, 24]. These methods require considerable expertise on the part of the human designer, and in recent years there has been much interest in replicating this success using reinforcement learning and imitation learning techniques [23, 18, 41, 55, 32] which could lower this burden, and perhaps also improve performance. The standard paradigm is to train an RL-based controller in a physics simulation environment and then transfer to the real world using various sim-to-real techniques [52, 40, 23]. This transfer has proven quite challenging, because the sim-to-real gap itself is the result of multiple factors: (a) the physical robot and its model in the simulator differ significantly; (b) real-world terrains vary considerably (Figure 1) from our models of these in the simulator; (c) the physics simulator fails to accurately capture the physics of the real world – we are dealing here with contact forces, deformable surfaces and the like – a considerably harder problem than modeling rigid bodies moving in free space.

In this paper, we report on our progress on solving this

A) Training in Simulation



B) Deployment

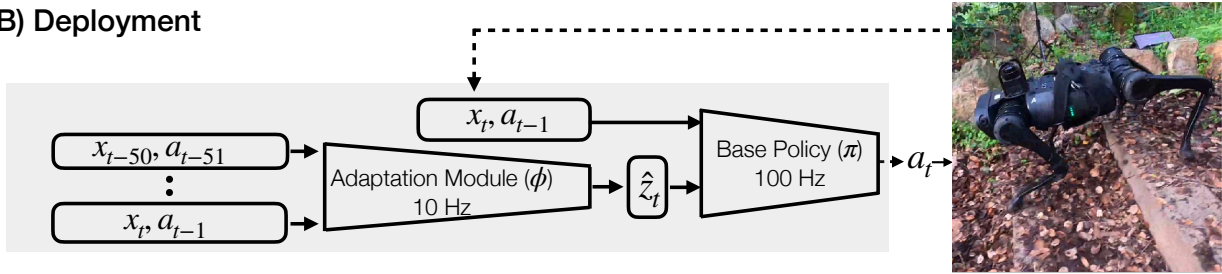


Fig. 2: RMA consists of two subsystems - the base policy π and the adaptation module ϕ . **Top:** RMA is trained in two phases. In the first phase, the base policy π takes as input the current state x_t , previous action a_{t-1} and the privileged environmental factors e_t , which is encoded into the latent extrinsics vector z_t using the environmental factor encoder μ . The base policy is trained in simulation using model-free RL. In the second phase, the adaptation module ϕ is trained to predict the extrinsics $\hat{z}_t$ from the history of state and actions via supervised learning with on-policy data. **Bottom:** At deployment, the adaptation module ϕ generates the extrinsics $\hat{z}_t$ at 10Hz, and the base policy generates the desired joint positions at 100Hz which are converted to torques using A1's PD controller. Since the adaptation module runs at a lower frequency, the base policy consumes the most recent extrinsics vector $\hat{z}_t$ predicted by the adaptation module to predict a_t . This asynchronous design was critical for seamless deployment on low-cost robots like A1 with limited on-board compute. Videos at: <https://ashish-kmr.github.io/rma-legged-robots/>

challenge for quadruped locomotion, using as an experimental platform the relatively cheap A1 robot from Unitree. Figure 1 shows some sample examples with in-action results in the video. Before outlining our approach (Figure 2), we begin by noting that human walking in the real world entails rapid adaptation as we move on different soils, uphill or downhill, carrying loads, with rested or tired muscles, and coping with sprained ankles and the like. Let us focus on this as a central problem for legged robots as well, and call it **Rapid Motor Adaptation (RMA)**. We will posit that RMA has to occur online, at a time scale of fractions of a second, which implies that we have no time to carry out multiple experiments in the physical world, rolling out multiple trajectories and optimizing to estimate various system parameters. It may be worse than that. If we introduce the quadruped onto a rocky surface with no prior experience, the robot policy would fail often, causing serious damage to the robot. Collecting even 3-5 mins of walking data in order to adapt the walking policy may be practically infeasible. Our strategy therefore entails that not just the basic

walking policy, but also RMA must be trained in simulation, and directly deployed in the real world. But, how?

Figure 2 shows that RMA consists of two subsystems: the base policy π and the adaptation module ϕ , which work together to enable online real time adaptation on a very diverse set of environment configurations. The base policy is trained via reinforcement learning in simulation using privileged information about the environment configuration e_t such as friction, payload, etc. Knowledge of the vector e_t allows the base policy to appropriately adapt to the given environment. The environment configuration vector e_t is first encoded into a latent feature space z_t using an encoder network μ . This latent vector z_t , which we call the *extrinsics*, is then fed into the base policy along with the current state x_t and the previous action a_{t-1} . The base policy then predicts the desired joint positions of the robot a_t . The policy π and the environmental factor encoder μ are jointly trained via RL in simulation.

Unfortunately, this policy cannot be directly deployed because we don't have access to e_t in the real world. What

we need to do is to estimate the extrinsics at run time, which is the role of the adaptation module ϕ . The key insight is that when we command a certain movement of the robot joints, the actual movement differs from that in a way that depends on the extrinsics. So instead of using privileged information, we might hope to use the recent history of the agent’s state to estimate this extrinsics vector, analogously to the operation of a Kalman filter for state estimation from history of observables. Specifically, the goal of ϕ is to estimate the extrinsics vector z_t from the robot’s recent state and action history, without assuming any access to e_t . That is at runtime, but at training time, life is easier. **Since both the state history and the extrinsics vector z_t can be computed in simulation, we can train this module via supervised learning.** At deployment, both these modules work together to perform robust and adaptive locomotion. In our experimental setup with its limited on-board computing, the base policy π runs at 100 Hz, while the adaptation module ϕ is slower and runs at 10Hz. The two run asynchronously in parallel with no central clock to align them. The base policy just uploads the most recent prediction of the extrinsics vector z_t from the adaptation module to predict action a_t .

Our approach is in contrast to previous learning-based work in locomotion that adapt learned policies via inferring the key parameters about the environment from a small dataset collected in every new situation to which the robot is introduced. These could either be physical parameters like friction, etc. [7] or their latent encoding [41]. Unfortunately, as mentioned earlier, collecting such a dataset, when the robot hasn’t yet acquired a good policy for walking, could result in falls and damage to the robot. Our approach avoids this because RMA, through the rapid estimation of z_t permits the walking policy to adapt quickly¹ and avoid falls.

Training of a base policy using RL with an extra argument for the environmental parameters has also been pursued in [57, 41]. Our novel aspects are the use of a varied terrain generator and “natural” reward functions motivated by bioenergetics which allows us to learn walking policies without using any reference demonstrations [41]. But the truly novel contribution of this paper is the adaptation module, trained in simulation, which makes RMA possible. This, at deployment time, has the flavor of system identification, but it is an on-line version of system identification, based just on the single trajectory that the robot has seen in the past fraction of a second. One might reasonably ask why it should work at all, but we can offer a few speculations:

- **System identification** is traditionally thought of as an optimization problem. But in many settings researchers have found that given sample (input, output) pairs of optimization problems with their solutions, we could use a neural network to approximate the function mapping the problem to its solution [1, 17]. Effectively that is what ϕ is learning to do.
- We don’t need perfect system identification for the

approach to work. The vector of extrinsics z_t is a lower-dimensional nonlinear projection of the environmental parameters. This takes care of some identifiability issues where some parameters could covary with identical effects on observables. Secondly, we don’t need this vector of extrinsics to be correct in some “ground truth” sense. What matters is that it leads to the “right” action, and the end-to-end training optimizes for that.

- The range of situations seen in training should encompass what the robot will encounter in the real world. We use a fractal terrain generator which accompanied by the randomization of parameters such as mass, friction etc. creates a wide variety of physical contexts in which the walking robot has to react.

The most comparable work in terms of robust performance of RL policies for legged locomotion in the real-world is that of Lee et al. [32] which, unlike our work, relies on hand-coded domain knowledge of predefined trajectory generator [25] and motor models [23]. We evaluated RMA across a wide variety of terrains in the real world (Figure 1). The proposed adaptive controller is able to walk on slippery surfaces, uneven ground, deformable surfaces (such as foam, mattress, etc) and on rough terrain in natural environments such as grass, long vegetation, concrete, pebbles, rocky surfaces, sand, etc.

II. RELATED WORK

Conventionally, legged locomotion has been approached by using control-based methods [36, 43, 16, 56, 50, 26, 28, 2, 24, 4]. MIT Cheetah 3 [5] can achieve high speed and jump over obstacles by using regularized model predictive control (MPC) and simplified dynamics [12]. The ANYmal robot [20] locomotes by optimizing a parameterized controller and planning based on an inverted pendulum model [15]. However, these methods require accurate modeling of the real-world dynamics, in-depth prior knowledge of the robots, and manual tuning of gaits and behaviors. Optimizing controllers, combined with MPC, can mitigate some of the problems [30, 8, 9], however they still require significant task-specific feature engineering [11, 15, 3].

Learning for Legged Locomotion Some of the earliest attempts to incorporate learning into locomotion can be dated back to DARPA Learning Locomotion Program [63, 64, 44, 62, 27]. More recently, deep reinforcement learning (RL) offered an alternative to alleviate the reliance on human expertise and has shown good results in simulation [48, 33, 37, 14]. However, such policies are difficult to transfer to the real world [31, 39, 6]. One approach is to directly train in the real world [18, 55]. However, such policies are limited to very simple setups, and scaling to complex setups requires unsafe exploration and a large number of samples.

Sim-to-Real Reinforcement Learning To achieve complex walking behaviours in the real world using RL, several methods try to bridge the Sim-to-Real gap. Domain randomization is a class of methods in which the policy is trained with a wide range of environment parameters and sensor noises to learn

¹RMA takes less than 1s, whereas Peng et al. [41] need to collect 4–8mins (50 episodes of 5 – 10s) of data.

behaviours which are robust in this range [51, 52, 40, 54, 38]. However, domain randomization trades optimality for robustness leading to an over conservative policy [34].

Alternately, the Sim-to-Real gap can also be reduced by making the simulation more accurate [23, 51, 19]. Tan et al. [51] improve the motor models by fitting a piece-wise linear function to data from the actual motors [51]. Hwangbo et al. [23], instead, use a neural network to parameterize the actuator model [23, 32]. However, these approaches require initial data collection from the robot to fit the motor model, and would require this to be done for every new setup.

System Identification and Adaptation Instead of being agnostic to physics parameters, the policy can condition on these parameters via online system identification. During deployment in the real world, physics parameters can either be inferred through a module that is trained in simulation [57], or be directly optimized for high returns by using evolutionary algorithms [58]. Predicting the exact system parameters is often unnecessary and difficult, leading to poor performance in practice. Instead, a low dimensional latent embedding can be used [41, 61]. At test time, this latent can be optimized using real-world rollouts by using policy gradient methods [41], Bayesian optimization [59], or random search [60]. Another approach is to use meta learning to learn an initialization of policy network for fast online adaptation [13]. Although they have been demonstrated on real robots [49, 10], they still require multiple real-world rollouts to adapt.

III. RAPID MOTOR ADAPTATION

We now describe each component of the RMA algorithm introduced in the third paragraph of Section I and summarized in Figure 2. Following sections discuss the base policy, the adaptation module and the deployment on the real-robot in order. We will use the same notation as introduced in Section I.

A. Base Policy

We learn a base policy π which takes as input the current state $x_t \in \mathbb{R}^{30}$, previous action $a_{t-1} \in \mathbb{R}^{12}$ and the extrinsics vector $z_t \in \mathbb{R}^8$ to predict the next action a_t . The predicted action a_t is the desired joint position for the 12 robot joints which is converted to torque using a PD controller. The extrinsics vector z_t is a low dimensional encoding of the environment vector $e_t \in \mathbb{R}^{17}$ generated by μ .

$$z_t = \mu(e_t) \quad (1)$$

$$a_t = \pi(x_t, a_{t-1}, z_t) \quad (2)$$

We implement μ and π as MLPs (details in Section IV-B). We jointly train the base policy π and the environmental factor encoder μ end to end using model-free reinforcement learning. At time step t , π takes the current state x_t , previous action a_{t-1} and the extrinsics $z_t = \mu(e_t)$, to predict an action a_t . RL maximizes the following expected return of the policy π :

$$J(\pi) = \mathbb{E}_{\tau \sim p(\tau|\pi)} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right],$$

where $\tau = \{(x_0, a_0, r_0), (x_1, a_1, r_1) \dots\}$ is the trajectory of the agent when executing policy π , and $p(\tau|\pi)$ represents the likelihood of the trajectory under π .

Stable Gait through Natural Constraints: Instead of adding artificial simulation noise, we train our agent under the following natural constraints. First, the reward function is motivated from bioenergetic constraints of minimizing work and ground impact [42]. We found these reward functions to be critical for learning realistic gaits in simulation. Second, we train our policies on uneven terrain (Figure 2) as a substitute for additional rewards used by [23] for foot clearance and robustness to external push. A walking policy trained under these natural constraints transfers to simple setups in the real world (like concrete or wooden floor) without any modifications. This is in contrast to other sim-to-real work which either calibrates the simulation with the real world [51, 23], or fine-tunes the policy in the real world [41]. The adaptation module then enables it to scale from simple setups to very challenging terrains as shown in Figure 1.

RL Rewards: The reward function encourages the agent to move forward with a maximum speed of 0.35 m/s, and penalizes it for jerky and inefficient motions. Let's denote the linear velocity as $\mathbf{v}$, the orientation as θ and the angular velocity as ω , all in the robot's base frame. We additionally define the joint angles as $\mathbf{q}$, joint velocities as $\dot{\mathbf{q}}$, joint torques as $\boldsymbol{\tau}$, ground reaction forces at the feet as $\mathbf{f}$, velocity of the feet as $\mathbf{v}_f$ and the binary foot contact indicator vector as $\mathbf{g}$. The reward at time t is defined as the sum of the following quantities:

- 1) Forward: $\min(v_x^t, 0.35)$
- 2) Lateral Movement and Rotation: $-\|\mathbf{v}_y^t\|^2 - \|\omega_{yaw}^t\|^2$
- 3) Work: $-\|\boldsymbol{\tau}^T \cdot (\mathbf{q}^t - \mathbf{q}^{t-1})\|$
- 4) Ground Impact: $-\|\mathbf{f}^t - \mathbf{f}^{t-1}\|^2$
- 5) Smoothness: $-\|\boldsymbol{\tau}^t - \boldsymbol{\tau}^{t-1}\|^2$
- 6) Action Magnitude: $-\|\mathbf{a}^t\|^2$
- 7) Joint Speed: $-\|\dot{\mathbf{q}}^t\|^2$
- 8) Orientation: $-\|\boldsymbol{\theta}_{roll, pitch}^t\|^2$
- 9) Z Acceleration: $-\|v_z^t\|^2$
- 10) Foot Slip: $-\|\text{diag}(\mathbf{g}^t) \cdot \mathbf{v}_f^t\|^2$

The scaling factor of each reward term is 20, 21, 0.002, 0.02, 0.001, 0.07, 0.002, 1.5, 2.0, 0.8 respectively.

Training Curriculum: If we naively train our agent with the above reward function, it learns to stay in place because of the penalty terms on the movement of the joints. To prevent this collapse, we follow the strategy described in [23]. We start the training with very small penalty coefficients, and then gradually increase the strength of these coefficients using a fixed curriculum. We also linearly increase the difficulty of other perturbations such as mass, friction and motor strength as the training progresses. We don't have any curriculum on the terrains and start the training with randomly sampling the terrain profiles from the same fixed difficulty.

B. Adaptation Module

The knowledge of privileged environment configuration e_t and its encoded extrinsics vector z_t are not accessible during deployment in the real-world. Hence, we propose to estimate the extrinsics online using the adaptation module ϕ . Instead of e_t , the adaptation module uses the recent history of robot's states $x_{t-k:t-1}$ and actions $a_{t-k:t-1}$ to generate $\hat{z}_t$ which is an estimate of the true extrinsics vector z_t . In our experiments, we use $k = 50$ which corresponds to 0.5s.

$$\hat{z}_t = \phi(x_{t-k:t-1}, a_{t-k:t-1})$$

Note that instead of predicting e_t , which is the case in typical system identification, we directly estimate the extrinsics z_t that only encodes how the behavior should change to correct for the given environment vector e_t .

To train the adaptation module, we just need the state-action history and the target value of z_t (given by the environmental factor encoder μ). Both of these are available in simulation, and hence, ϕ can be trained via supervised learning to minimize: $\text{MSE}(\hat{z}_t, z_t) = \|\hat{z}_t - z_t\|^2$, where $z_t = \mu(e_t)$. We model ϕ as a 1-D CNN to capture temporal correlations (Section IV-B).

One way to collect the state-action history is to unroll the trained base policy π with the ground truth z_t . However, such a dataset will contain examples of only good trajectories where the robot walks seamlessly. Adaptation module ϕ trained on this data would not be robust to deviations from the expert trajectory, which will happen often during deployment.

We resolve this problem by training ϕ with on-policy data (similar to Ross et al. [45]). We unroll the base policy π with the $\hat{z}_t$ predicted by the randomly initialized policy ϕ . We then use this state action history, paired with the *ground truth* z_t to train ϕ . We iteratively repeat this until convergence. This training procedure ensures that RMA sees enough exploration trajectories during training due to *a)* randomly initialized ϕ , and *b)* imperfect prediction of $\hat{z}_t$. This adds robustness to the performance of RMA during deployment.

C. Asynchronous Deployment

We train RMA completely in simulation and then deploy it in the real world without any modification or fine-tuning. The two subsystems of RMA run asynchronously and at substantially different frequencies, and hence, can easily run using little on-board compute. The adaptation policy is slow because it operates on the state-action history of 50 time steps, roughly updating the extrinsic vector $\hat{z}_t$ once every 0.1s (10 Hz). The base policy runs at 100 Hz and uses the most recent $\hat{z}_t$ generated by the adaptation module, along with the current state and the previous action, to predict a_t . This asynchronous execution doesn't hurt performance in practice because $\hat{z}_t$ changes relatively infrequently in the real world.

Alternately, we could have trained a base policy which directly takes the state and action history as input without decoupling them into the two modules. We found that this (a) leads to unnatural gaits and poor performance in simulation, (b) can only run at 10Hz on the on-board compute, and (c) lacks the asynchronous design which is critical for a seamless

Parameters	Training Range	Testing Range
Friction	[0.05, 4.5]	[0.04, 6.0]
K_p	[50, 60]	[45, 65]
K_d	[0.4, 0.8]	[0.3, 0.9]
Payload (Kg)	[0, 6]	[0, 7]
Center of Mass (cm)	[-0.15, 0.15]	[-0.18, 0.18]
Motor Strength	[0.90, 1.10]	[0.88, 1.22]
Re-sample Probability	0.004	0.01

TABLE I: Ranges of the environmental parameters.

deployment of RMA on the real robot without the need for any synchronization or calibration of the two subsystems. This asynchronous design is fundamentally enabled by the decoupling of the relatively infrequently changing extrinsics vector with the quickly changing robot state.

IV. EXPERIMENTAL SETUP

A. Environment Details

Hardware Details: We use A1 robot from Unitree for all our real-world experiments. A1 is a relatively low cost medium sized robotic quadruped dog. It has 18 degrees of freedom out of which 12 are actuated (3 motors on each leg) and weighs about 12 kg. To measure the current state of the robot, we use the joint position and velocity from the motor encoders, roll and pitch from the IMU sensor and the binarized foot contact indicators from the foot sensors. The deployed policy uses position control for the joints of the robots. The predicted desired joint positions are converted to torque using a PD controller with fixed gains ($K_p = 55$ and $K_d = 0.8$).

Simulation Setup: We use the RaiSim simulator [22] for rigid-body and contact dynamics simulation. We import the A1 URDF file from Unitree [53] and use the inbuilt fractal terrain generator to generate uneven terrain (fractal octaves = 2, fractal lacunarity = 2.0, fractal gain = 0.25, z-scale = 0.27). Each RL episode lasts for a maximum of 1000 steps, with early termination if the height of the robots drops below 0.28m, magnitude of the body roll exceeds 0.4 radians or the pitch exceeds 0.2 radians. The control frequency of the policy is 100 Hz, and the simulation time step is 0.025s.

State-Action Space: The state is 30 dimensional containing the joint positions (12 values), joint velocities (12 values), roll and pitch of the torso and binary foot contact indicators (4 values). For actions, we use position control for the 12 robot joints. RMA predicts the desired joint angles $a = \hat{\mathbf{q}} \in \mathbb{R}^{12}$, which is converted to torques τ using a PD controller: $\tau = K_p (\hat{\mathbf{q}} - \mathbf{q}) + K_d (\dot{\hat{\mathbf{q}}} - \dot{\mathbf{q}})$. K_p and K_d are manually-specified gains, and the target joint velocities $\dot{\hat{\mathbf{q}}}$ are set to 0.

Environmental Variations: All environmental variations with their ranges are listed in Table I. Of these, e_t includes mass and its position on the robot (3 dims), motor strength (12 dims), friction (scalar) and local terrain height (scalar), making it a 17-dim vector. Note that although the difficulty of the terrain profile is fixed, the local terrain height changes as the agent

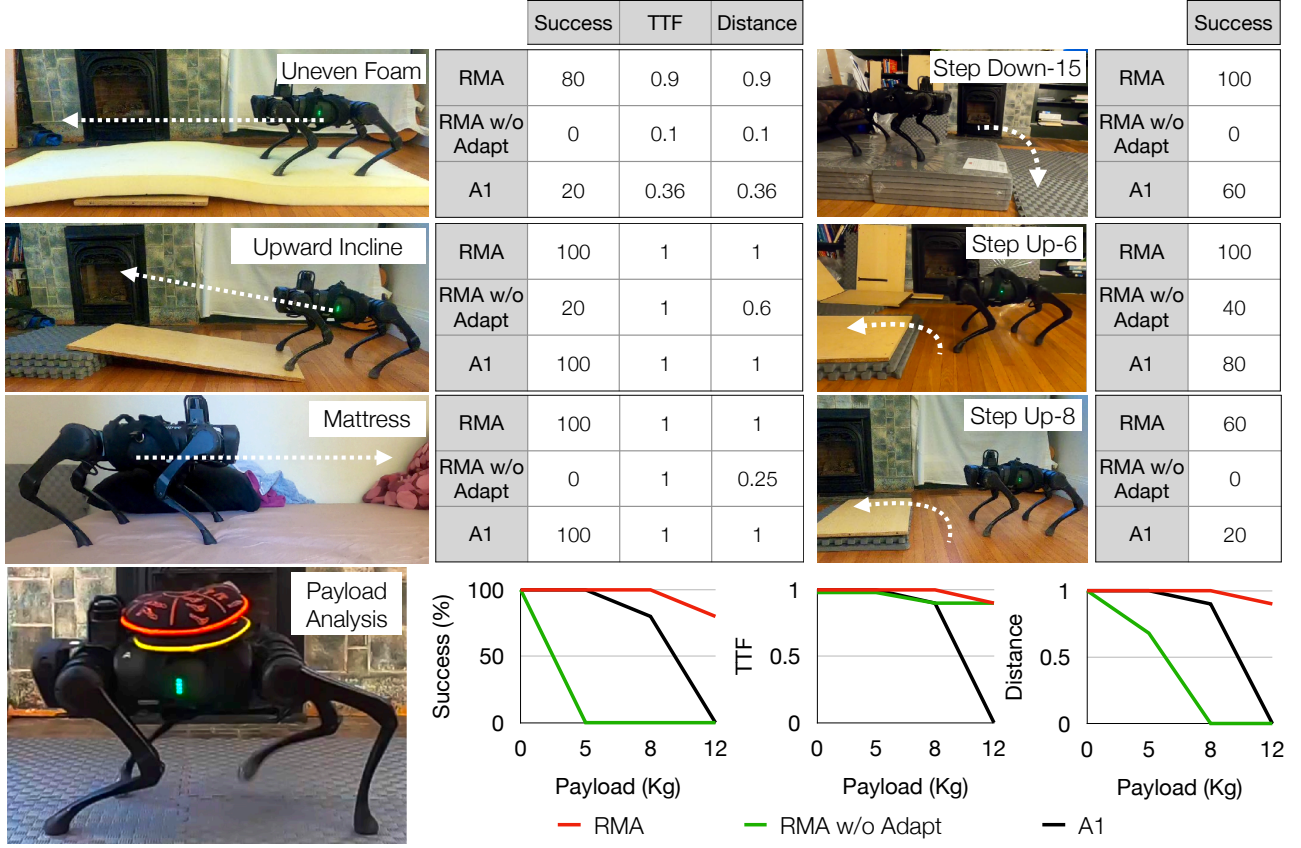


Fig. 3: We evaluate RMA in several out-of-distribution setups in the real world. We compare RMA to A1’s controller and RMA without the adaptation module. We find that RMA steps down a height of 15cm with 80% success rate and walks over unseen deformable surfaces, such as a memory foam mattress and a slightly uneven foam with 100% success rate. It is also able to successfully climb inclines and steps. A1’s controller fails to walk over uneven foam. At the bottom, we also analyze the payload carrying limits of the three methods. We see that the A1 controller’s performance starts degrading at 8Kg payload capacity. RMA w/o adaptation fails to move for payloads more than 8Kg, but rarely falls. For reference, A1 robot weights 12Kg. Overall, the proposed method consistently dominates the baseline methods. The numbers reported are averaged over 5 trials.

moves. We discretize the terrain height under each foot to the first decimal place and then take the maximum among the four feet to get a scalar. This ensures that the controller does not critically depend on a fast and accurate sensing of the local terrain, and allows the base policy to use it asynchronously at a much lower update frequency during deployment.

B. Training Details

Base Policy and Environment Factor Encoder Architecture: The base policy is a 3-layer multi-layer perceptron (MLP) which takes in the current state $x_t \in \mathbb{R}^{30}$, previous action $a_{t-1} \in \mathbb{R}^{12}$ and the extrinsics vector $z_t \in \mathbb{R}^8$, and outputs 12-dim target joint angles. The dimension of hidden layers is 128. The environment factor encoder is a 3-layer MLP (256, 128 hidden layer sizes) and encodes $e_t \in \mathbb{R}^{17}$ into $z_t \in \mathbb{R}^8$.

Adaptation Module Architecture: The adaptation module first embeds the recent states and actions into 32-dim representations using a 2-layer MLP. Then, a 3-layer 1-D CNN convolves the representations across the time dimension to capture temporal correlations in the input. The input channel

number, output channel number, kernel size, and stride of each layer are $[32, 32, 8, 4]$, $[32, 32, 5, 1]$, $[32, 32, 5, 1]$. The flattened CNN output is linearly projected to estimate $\hat{z}_t$.

Learning Base Policy and Environmental Factor Encoder Network: We jointly train the base policy and the environment encoder network using PPO [48] for 15,000 iterations each of which uses batch size of 80,000 split into 4 mini-batches. The learning rate is set to $5e-4$. The coefficient of the reward terms are provided in Section III. Training takes roughly 24 hours on an ordinary desktop machine, with 1 GPU for policy training. In this duration, it simulates 1.2 billion steps.

Learning Adaptation Module: We train the adaptation module using supervised learning with on-policy data. We use Adam optimizer [29] to minimize MSE loss. We run the optimization process for 1000 iterations with a learning rate of $5e-4$ each of which uses a batch size of 80,000 split up into 4 mini-batches. It takes 3 hours to train this on an ordinary desktop machine, with 1 GPU for training the policy. In this duration, it simulates 80 million steps.

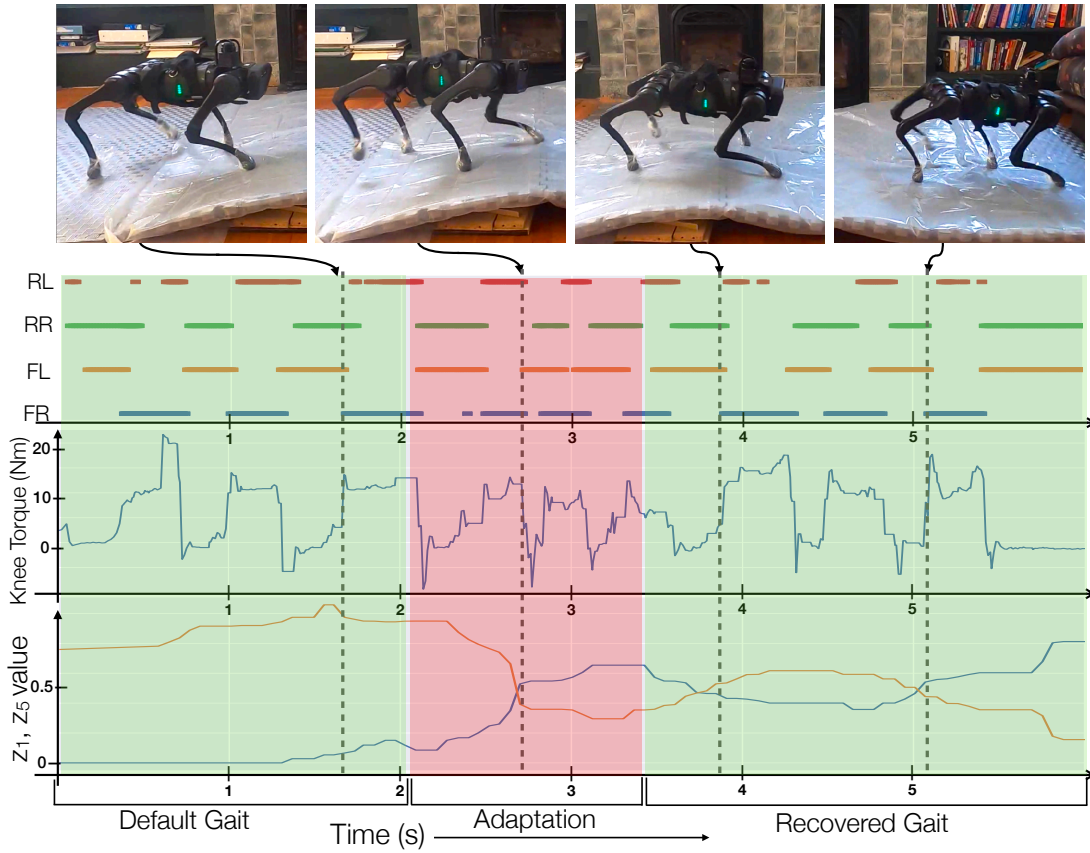


Fig. 4: We analyze RMA as the robot walks over an oily plastic sheet with additional plastic covering on its feet. We plot the torque of the knee and the gait pattern which indicates the contact of the four feet (F/R denotes Front/Rear and R/L denotes Right/Left). The bottom plot shows median filtered 1st and 5th components of the extrinsics vector $\hat{z}$ predicted by the adaptation module. When the robot enters the slippery patch we see a change in the two components of the extrinsics vector $\hat{z}$, indicating that **the slip event has been detected by the adaptation module**. Note that post adaptation, the recovered gait time period is similar to the original, the torque magnitudes have increased and $\hat{z}$ continues to capture the fact that the surface is still slippery. RMA was successful in 90% of the runs over oily patch.

V. RESULTS AND ANALYSIS

We compare the performance of RMA with several baselines in simulation (Table II). We additionally compare to the manufacturer’s controller, which ships with A1, in the real world indoor setups (Figure 3) and run RMA in the wild in a very diverse set of terrains (Figure 1). Videos at <https://ashish-kmr.github.io/rma-legged-robots/>

Baselines: We compare to the following baselines:

- 1) A1 Controller: The default robot manufacturer’s controller which uses a force-based control scheme with MPC.
- 2) Robustness through Domain Randomization (Robust): The base policy is trained without z_t to be robust to the variations in the training range [52, 40].
- 3) Expert Adaptation Policy (Expert): In simulation, we can use the true value of the extrinsics vector z_t . This is an upper bound to the performance of RMA.
- 4) RMA w/o Adaptation: We can also evaluate the performance of the base policy without the adaptation module to ablate the importance of the adaptation module.

5) System Identification [57]: Instead of predicting $\hat{z}_t$, we directly predict the system parameters $\hat{c}_t$.

6) Advantage Weighted Regression for Domain Adaptation (AWR) [41]: Optimize $\hat{z}_t$ offline using AWR by using real-world rollouts of the policy in the testing environment.

Learning baselines were trained with the same architecture, reward function and other hyper-parameters.

Metrics: We compare the performance of RMA against baselines using the following metrics: (1) time-to-fall divided by maximum episode length to get a normalized value between 0 – 1 (TTF); (2) average forward reward, (3) success rate, (4) distance covered, (5) exploration samples needed for adaptation, (6) torque applied, (7) smoothness which is derivative of torque and (7) ground impact (details in the supplementary).

A. Indoor Experiments

In the real world, we compare RMA with A1’s controller and with RMA without the adaptation module (Figure 3). We limit comparison to these two baselines to avoid damage to the

	Success (%)	TTF	Reward	Distance (m)	Samples	Torque	Smoothness	Ground Impact
Robust [52, 40]	62.4	0.80	4.62	1.13	0	527.59	122.50	4.20
SysID [57]	56.5	0.74	4.82	1.17	0	565.85	149.75	4.03
AWR [41]	41.7	0.65	4.17	0.95	40k	599.71	162.60	4.02
RMA w/o Adapt	52.1	0.75	4.72	1.15	0	524.18	106.25	4.55
RMA	73.5	0.85	5.22	1.34	0	500.00	92.85	4.27
Expert	76.2	0.86	5.23	1.35	0	485.07	85.56	3.90

TABLE II: **Simulation Testing Results:** We compare the performance of our method to baseline methods in simulation. Our train and test settings are listed in Table I. We resample the environment parameters within an episode with a re-sampling probability of 0.01 per step during testing. Baselines and metrics are defined in Section V. The numbers reported are averaged over 3 randomly initialized policies and 1000 episodes per random initialization. RMA beats the performance of all the baselines, with only a slight degradation in performance compared to the Expert.

robot hardware. We run 5 trials for each method and report the success rate, time to fall (TTF), and distance covered. Note that if a method drastically failed at a task, we only run two trials and then report a failure. This is done to minimize damage to the robot hardware. We have the following indoor setups:

- **n-kg Payload:** Walk 300cm with n-kg payload on top.
- **StepUp-n:** Step up on an n-cm high step.
- **Uneven Foam:** Walk 180cm on a center elevated foam.
- **Mattress:** Walk 60cm on a memory foam mattress.
- **StepDown-n:** Step down on an n-cm high step.
- **Incline:** Walk up on a 6-degrees incline.
- **Oily Surface:** Cross through an oily patch.

Each trial of **StepUp-n** and **StepDown-n** is terminated after a success or a failure. Thus, we only report the success rate for these tasks because other metrics are meaningless.

We observe that RMA achieves a high success rate in all these setups, beating the performance of A1’s controller by a large margin in some cases. We find that turning off the adaptation module substantially degrades performance, implying that the adaptation module is critical to solve these tasks. A1’s controller struggled with uneven foam and with a large step down and step up. The controller was destabilized by unstable footholds in most of its failures. In the payload analysis, the A1’s controller was able to handle higher than the advertised payload (5Kg), but starts sagging, and eventually falls as the payload increases. In contrast, RMA maintains the height and is able to carry up to 12Kg (100% of body weight) with a high success rate. RMA w/o adaptation mostly doesn’t fall, but also doesn’t move forward. We also evaluated RMA in a more challenging task of crossing an oily path with plastic wrapped feet. The robot successfully walks across the oily patch. Interestingly, RMA w/o adaptation was able to walk successfully on wooden floor without any fine-tuning or simulation calibration. This is in contrast to existing methods which calibrate the simulation [51, 23] or fine-tune their policy at test time [41] even for flat and static environments.

B. Outdoor Experiments

We demonstrate the performance of RMA on several challenging outdoor environments as shown in Figure 1. The

robot is successfully able to walk on sand, mud and dirt without a single failure in all our trials. These terrains make locomotion difficult due to sinking and sticking feet, which requires the robot to change the footholds dynamically to ensure stability. RMA had a 100% success rate for walking on tall vegetation or crossing a bush. Such terrains obstruct the feet of the robot, making it periodically unstable as it walks. To successfully walk in these setups, the robot has to stabilize against foot entanglements, and power through some of these obstructions aggressively. We also evaluate our robot on walking down some stairs found on a hiking trail. The robot was successful in 70% of the trials, which is still remarkable given that the robot never sees a staircase during training. And lastly, we test the robot over construction debris, where it was successful 100% of the times when walking downhill over a mud pile and 80% of the times when walking across a cement pile and a pile of pebbles. The cement pile and pebbles were itself on a ground which was steeply sloping sideways, making it very challenging for the robot to go across the pile.

C. Simulation Results

We compare the performance of our method to baseline methods in simulation (Table II). We sample our training and testing parameters according to Table I, and resample them within an episode with a resampling probability of 0.004 and 0.01 per step respectively for training and testing. The numbers reported are averaged over 3 randomly initialized policies and 1000 episodes per random initialization. RMA performs the best with only a slight degradation compared to Expert’s performance. The constantly changing environment leads to poor performance of AWR which is very slow to adapt. Since the Robust baseline is agnostic to extrinsics, it learns a very conservative policy which loses on performance. Note that the low performance of SysID implies that explicitly estimating e_t is difficult and unnecessary to achieve superior performance. We also compare to RMA w/o adaptation, which shows a significant performance drop without the adaption module.

D. Adaptation Analysis

We analyze the gait patterns, torque profiles and the estimated extrinsics vector $\hat{z}_t$ for adaptation over slippery surface (Figure

4). We pour oil on the plastic surface on the ground and additionally cover the feet of the robot in plastic. The robot then tries to cross the slippery patch and is able to successfully adapt to it. We found that RMA was successful in 90% of the runs over oily patch. For one such trial, we plot the torque profile of the knee, the gait pattern, and median filtered 1st and 5th components of the extrinsics vector $\hat{z}_t$ in Figure 4. When the robot first starts slipping somewhere around 2s, the slip disturbs the regular motion of the robot, after which it enters the adaptation phase. This is noticeable in the plotted components of the extrinsics vector which change in response to the slip. This detected slip enables the robot to recover and continue walking over the slippery patch. Note that although post adaptation, the torque stabilizes to a slightly higher magnitude and the gait time period is roughly recovered, the extrinsics vector does not recover and continues to capture the fact that the surface is slippery. See supplementary more such analysis.

VI. CONCLUSION

We presented the RMA algorithm for real-time adaptation of a legged robot walking in a variety of terrains. **No demonstrations or predefined motion templates were needed.** Despite only having access to proprioceptive data, the robot can also go downstairs and walk across rocks. However, a blind robot has limitations. Larger perturbations such as sudden falls while going downstairs, or due to multiple leg obstructions from rocks, sometimes lead to failures. To develop a truly reliable walking robot, **we need to use not just proprioception but also exteroception with an onboard vision sensor.** The importance of vision in guiding long range, rapid locomotion has been well studied, e.g. by [35], and this is an important direction for future work.

ACKNOWLEDGMENTS

We would like to thank Jemin Hwangbo for helping with the simulation platform, and Koushil Sreenath and Stuart Anderson for helpful feedback during the course of this project. We would also like to thank Claire Tomlin, Shankar Sastry, Chris Atkeson, Aravind Sivakumar, Ilija Radosavovic and Russell Mendonca for their high quality feedback on the paper. This research was part of a BAIR-FAIR collaborative project, and recently also supported by the DARPA Machine Common Sense program.

REFERENCES

- [1] MYM Ahmed and N Qin. Surrogate-based aerodynamic design optimization: Use of surrogates in aerodynamic design optimization. In *International Conference on Aerospace Sciences and Aviation Technology*, 2009. 3
- [2] Aaron D Ames, Kevin Galloway, Koushil Sreenath, and Jessie W Grizzle. Rapidly exponentially stabilizing control lyapunov functions and hybrid zero dynamics. *IEEE Transactions on Automatic Control*, 2014. 1, 3
- [3] Taylor Apgar, Patrick Clary, Kevin Green, Alan Fern, and Jonathan W Hurst. Fast online trajectory optimization

- for the bipedal robot cassie. In *Robotics: Science and Systems*, 2018. 3
- [4] Monica Barragan, Nikolai Flowers, and Aaron M. Johnson. MiniRHex: A small, open-source, fully programmable walking hexapod. In *Robotics: Science and Systems Workshop on “Design and Control of Small Legged Robots”*, 2018. 3
- [5] Gerardo Bledt, Matthew J Powell, Benjamin Katz, Jared Di Carlo, Patrick M Wensing, and Sangbae Kim. Mit cheetah 3: Design and control of a robust, dynamic quadruped robot. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018. 3
- [6] Adrian Boeing and Thomas Bräunl. Leveraging multiple simulators for crossing the reality gap. In *2012 12th International Conference on Control Automation Robotics & Vision (ICARCV)*. IEEE, 2012. 3
- [7] Josh C Bongard and Hod Lipson. Nonlinear system identification using coevolution of models and tests. *IEEE Transactions on Evolutionary Computation*, 2005. 3
- [8] Roberto Calandra, André Seyfarth, Jan Peters, and Marc Peter Deisenroth. Bayesian optimization for learning gaits under uncertainty. *Annals of Mathematics and Artificial Intelligence*, 2016. 3
- [9] Krzysztof Choromanski, Atil Iscen, Vikas Sindhwani, Jie Tan, and Erwin Coumans. Optimizing simulations with noise-tolerant structured exploration. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018. 3
- [10] Ignasi Clavera, Anusha Nagabandi, Simin Liu, Ronald S. Fearing, Pieter Abbeel, Sergey Levine, and Chelsea Finn. Learning to adapt in dynamic, real-world environments through meta-reinforcement learning. In *International Conference on Learning Representations*, 2019. 4
- [11] Martin De Lasa, Igor Mordatch, and Aaron Hertzmann. Feature-based locomotion controllers. *ACM Transactions on Graphics (TOG)*, 2010. 3
- [12] Jared Di Carlo, Patrick M Wensing, Benjamin Katz, Gerardo Bledt, and Sangbae Kim. Dynamic locomotion in the mit cheetah 3 through convex model-predictive control. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018. 3
- [13] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning*. PMLR, 2017. 4
- [14] Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*. PMLR, 2018. 3
- [15] Christian Gehring, Stelian Coros, Marco Hutter, Carmine Dario Bellicoso, Huub Heijnen, Remo Diethelm, Michael Bloesch, Péter Fankhauser, Jemin Hwangbo, Mark Hoepflinger, et al. Practice makes perfect: An optimization-based approach to controlling agile motions for a quadruped robot. *IEEE Robotics & Automation Magazine*, 2016. 3

- [16] Hartmut Geyer, Andre Seyfarth, and Reinhard Blickhan. Positive force feedback in bouncing gaits? *Proceedings of the Royal Society of London. Series B: Biological Sciences*, 2003. 1, 3
- [17] Xiaoxiao Guo, Wei Li, and Francesco Iorio. Convolutional neural networks for steady flow approximation. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016. 3
- [18] Tuomas Haarnoja, Sehoon Ha, Aurick Zhou, Jie Tan, George Tucker, and Sergey Levine. Learning to walk via deep reinforcement learning. In *Robotics: Science and Systems*, 2019. 1, 3
- [19] Josiah Hanna and Peter Stone. Grounded action transformation for robot learning in simulation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2017. 4
- [20] Marco Hutter, Christian Gehring, Dominic Jud, Andreas Lauber, C Dario Bellicoso, Vassilios Tsounis, Jemin Hwangbo, Karen Bodie, Peter Fankhauser, Michael Bloesch, et al. Anymal-a highly mobile and dynamic quadrupedal robot. In *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2016. 3
- [21] Jemin Hwangbo. RaisimGymTorch. <https://raisim.com/sections/RaisimGymTorch.html>, 2020-2021. 12
- [22] Jemin Hwangbo, Joonho Lee, and Marco Hutter. Per-contact iteration method for solving contact dynamics. *IEEE Robotics and Automation Letters*, 2018. URL www.raisim.com. 5
- [23] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 2019. 1, 3, 4, 8, 12
- [24] Dong Jin Hyun, Jongwoo Lee, SangIn Park, and Sangbae Kim. Implementation of trot-to-gallop transition and subsequent gallop on the mit cheetah i. *The International Journal of Robotics Research*, 2016. 1, 3
- [25] Atil Iscen, Ken Caluwaerts, Jie Tan, Tingnan Zhang, Erwin Coumans, Vikas Sindhwani, and Vincent Vanhoucke. Policies modulating trajectory generators. In *Conference on Robot Learning*. PMLR, 2018. 3
- [26] Aaron M Johnson, Thomas Libby, Evan Chang-Siu, Masayoshi Tomizuka, Robert J Full, and Daniel E Koditschek. Tail assisted dynamic self righting. In *Adaptive Mobile Robotics*. World Scientific, 2012. 1, 3
- [27] Mrinal Kalakrishnan, Jonas Buchli, Peter Pastor, Michael Mistry, and Stefan Schaal. Fast, robust quadruped locomotion over challenging terrain. In *2010 IEEE International Conference on Robotics and Automation*. IEEE, 2010. 3
- [28] Mahdi Khoramshahi, Hamed Jalaly Bidgoly, Soroosh Shafiee, Ali Asaei, Auke Jan Ijspeert, and Majid Nili Ahmadabadi. Piecewise linear spine for speed–energy efficiency trade-off in quadruped robots. *Robotics and Autonomous Systems*, 2013. 1, 3
- [29] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations*, 2015. 6, 12
- [30] Jens Kober, J Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 2013. 3
- [31] Sylvain Koos, Jean-Baptiste Mouret, and Stéphane Doncieux. Crossing the reality gap in evolutionary robotics by promoting transferable controllers. In *Proceedings of the 12th annual conference on Genetic and evolutionary computation*, 2010. 3
- [32] Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 2020. 1, 3, 4
- [33] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *ICLR*, 2016. 3
- [34] Jingru Luo and Kris Hauser. Robust trajectory optimization under frictional contact with iterative learning. *Autonomous Robots*, 2017. 4
- [35] Jonathan Samir Matthis, Jacob L Yates, and Mary M Hayhoe. Gaze and the control of foot placement when walking in natural terrain. *Current Biology*, 2018. 9
- [36] Hirofumi Miura and Isao Shimoyama. Dynamic walk of a biped. *The International Journal of Robotics Research*, 1984. 1, 3
- [37] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*. PMLR, 2016. 3
- [38] Ofir Nachum, Michael Ahn, Hugo Ponte, Shixiang Shane Gu, and Vikash Kumar. Multi-agent manipulation via locomotion using hierarchical sim2real. In *Conference on Robot Learning*. PMLR, 2020. 4
- [39] Michael Neunert, Thiago Boaventura, and Jonas Buchli. Why off-the-shelf physics simulators fail in evaluating feedback controller performance—a case study for quadrupedal robots. In *Advances in Cooperative Robotics*. World Scientific, 2017. 3
- [40] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018. 1, 4, 7, 8
- [41] Xue Bin Peng, Erwin Coumans, Tingnan Zhang, Tsang-Wei Edward Lee, Jie Tan, and Sergey Levine. Learning agile robotic locomotion skills by imitating animals. In *Robotics: Science and Systems*, 2020. 1, 3, 4, 7, 8
- [42] Delyle T Polet and John EA Bertram. An inelastic quadrupedal model discovers four-beat walking, two-beat running, and pseudo-elastic actuation as energetically optimal. *PLoS computational biology*, 2019. 4

- [43] Marc H Raibert. Hopping in legged systems—modeling and simulation for the two-dimensional one-legged case. *IEEE Transactions on Systems, Man, and Cybernetics*, 1984. 1, 3
- [44] Nathan Ratliff, Matt Zucker, J Andrew Bagnell, and Siddhartha Srinivasa. Chomp: Gradient optimization techniques for efficient motion planning. In *2009 IEEE International Conference on Robotics and Automation*. IEEE, 2009. 3
- [45] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, 2011. 5
- [46] Uluc Saranli, Martin Buehler, and Daniel E Koditschek. Rhex: A simple and highly mobile hexapod robot. *The International Journal of Robotics Research*, 2001. 1
- [47] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *4th International Conference on Learning Representations*, 2016. 12
- [48] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 3, 6, 12
- [49] Xingyou Song, Yuxiang Yang, Krzysztof Choromanski, Ken Caluwaerts, Wenbo Gao, Chelsea Finn, and Jie Tan. Rapidly adaptable legged robots via evolutionary meta-learning. In *International Conference on Intelligent Robots and Systems (IROS)*, 2020. 4
- [50] Koushil Sreenath, Hae-Won Park, Ioannis Poulakakis, and Jessie W Grizzle. A compliant hybrid zero dynamics controller for stable, efficient and fast bipedal walking on mabel. *The International Journal of Robotics Research*, 2011. 1, 3
- [51] Jie Tan, Tingnan Zhang, Erwin Coumans, Atil Iscen, Yunfei Bai, Danijar Hafner, Steven Bohez, and Vincent Vanhoucke. Sim-to-real: Learning agile locomotion for quadruped robots. In *Robotics: Science and Systems*, 2018. 4, 8
- [52] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017. 1, 4, 7, 8
- [53] Xingxing Wang. Unitree Robotics. <https://www.unitree.com/>. 5
- [54] Zhaoming Xie, Xingye Da, Michiel van de Panne, Buck Babich, and Animesh Garg. Dynamics randomization revisited: A case study for quadrupedal locomotion. *arXiv preprint arXiv:2011.02404*, 2020. 4
- [55] Yuxiang Yang, Ken Caluwaerts, Atil Iscen, Tingnan Zhang, Jie Tan, and Vikas Sindhwani. Data efficient reinforcement learning for legged robots. In *Conference on Robot Learning*. PMLR, 2020. 1, 3
- [56] KangKang Yin, Kevin Loken, and Michiel Van de Panne. Simbicon: Simple biped locomotion control. *ACM Transactions on Graphics (TOG)*, 2007. 1, 3
- [57] Wenhao Yu, Jie Tan, C. Karen Liu, and Greg Turk. Preparing for the unknown: Learning a universal policy with online system identification. In *Robotics: Science and Systems*, 2017. 3, 4, 7, 8
- [58] Wenhao Yu, C Karen Liu, and Greg Turk. Policy transfer with strategy optimization. In *International Conference on Learning Representations*, 2018. 4
- [59] Wenhao Yu, Visak C. V. Kumar, Greg Turk, and C. Karen Liu. Sim-to-real transfer for biped locomotion. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2019. 4
- [60] Wenhao Yu, Jie Tan, Yunfei Bai, Erwin Coumans, and Sehoon Ha. Learning fast adaptation with meta strategy optimization. *IEEE Robotics and Automation Letters*, 2020. 4
- [61] Wenxuan Zhou, Lerrel Pinto, and Abhinav Gupta. Environment probing interaction policies. In *7th International Conference on Learning Representations, ICLR 2019*, 2019. 4
- [62] J Zico Kolter and Andrew Y Ng. The stanford littledog: A learning and rapid replanning approach to quadruped locomotion. *The International Journal of Robotics Research*, 2011. 3
- [63] Matt Zucker, J Andrew Bagnell, Christopher G Atkeson, and James Kuffner. An optimization approach to rough terrain locomotion. In *2010 IEEE International Conference on Robotics and Automation*. IEEE, 2010. 1, 3
- [64] Matt Zucker, Nathan Ratliff, Martin Stolle, Joel Chestnutt, J Andrew Bagnell, Christopher G Atkeson, and James Kuffner. Optimization and learning for rough terrain legged locomotion. *The International Journal of Robotics Research*, 2011. 3

Supplementary for RMA: Rapid Motor Adaptation for Legged Robots

S1. METRICS

We use several metrics (in SI units) to evaluate and compare the performance of RMA against baselines:

- Success Rate: Average rate of successfully completing the task as defined in the next section.
- Time to Fall (TTF): Measures the time before a fall. We divide it by the maximum duration of the episode and report a normalized value between 0 and 1.
- Reward: Average step forward reward plus lateral reward over multiple episodes as defined in Section III-A RL Rewards of the main paper.
- Distance: Average distance covered in an episode. For real-world experiments, we report the normalized distance, where we normalize by the maximum distance which is specific to the task.
- Adaptation Samples: Number of control steps to explore in the testing environment needed for the motor policy to adapt.
- Torque: Squared L2 norm of torques at every joint $\|\tau^t\|^2$.
- Jerk: Squared L2 norm of delta torques $\|\tau^t - \tau^{t-1}\|^2$.
- Ground Impact: Squared L2 norm of delta ground reaction forces at every foot $\|\mathbf{f}^t - \mathbf{f}^{t-1}\|^2$.

S1. ADDITIONAL TRAINING AND DEPLOYMENT DETAILS

The training pipeline is shown in Algorithm 1, and the deployment pipeline is shown in Algorithm 2.

We use PPO [48] to train the base policy and the environmental factor encoder. We train for total 15,000 iterations. During each iteration, we collect a batch of 80,000 state-action transitions, which is evenly divided into 4 mini-batches. Each mini-batch is fed into the base policy and the Environment Factor Encoder in sequence for 4 rounds to compute the loss and error back-propagation. The loss is the sum of surrogate policy loss and 0.5 times the value loss. We clip the action log probability ratios between 0.8 and 1.2, and clip the target values to be within the 0.8 – 1.2 times range of the corresponding old values. We exclude the entropy regularization of the base policy, but constrain the standard deviation of the parameterized Gaussian action space to be large than 0.2 to ensure exploration. λ and γ in the generalization advantage estimation [47] are set to 0.95 and 0.998 respectively. We use the Adam optimizer [29], where we set the learning rate to $5e-4$, β to (0.9, 0.999), and ϵ to $1e-8$. The reference implementation can be found in the RaisimGymTorch Library [21].

If we naively train our agent with the reward function aggregating all the terms, it learns to fall because of the penalty terms. To prevent this collapse, we follow the strategy described in [23]. In addition the scaling factors of all reward terms, we apply a small multiplier k_t to the penalty terms 3 – 10, as defined in Section III-A of the main paper. We start the training

Algorithm 1: Rapid Motor Adaptation Training

Phase 1 Randomly initialize the base policy π ;
Randomly initialize the environmental factor encoder μ ; Empty replay buffer D_1 ;
for $0 \leq \text{itr} \leq N_{\text{itr}}^1$ **do**
 for $0 \leq i \leq N_{\text{env}}$ **do**
 $x_0, e_0 \leftarrow \text{envs}[i].\text{reset}()$;
 for $0 \leq t \leq T$ **do**
 $z_t \leftarrow \mu(e_t)$;
 $a_t \leftarrow \pi(x_t, a_{t-1}, z_t)$;
 $x_{t+1}, e_{t+1}, r_t \leftarrow \text{envs}[i].\text{step}(a_t)$;
 Store $((x_t, e_t), a_t, r_t, (x_{t+1}, e_{t+1}))$ in D_1 ;
 end
 end
 Update π and μ using PPO [48];
 Empty D_1 ;
end

Phase 2 Randomly initialize the adaptation module ϕ
parameterized by θ_ϕ ; Empty mini-batch D_2 ;
for $0 \leq \text{itr} \leq N_{\text{itr}}^2$ **do**
 for $0 \leq i \leq N_{\text{env}}$ **do**
 $x_0, e_0 \leftarrow \text{envs}[i].\text{reset}()$;
 for $0 \leq t \leq T$ **do**
 $\hat{z}_t \leftarrow \phi(x_{t-k:k}, a_{t-k-1:k-1})$;
 $z_t \leftarrow \mu(e_t)$;
 $a_t \leftarrow \pi(x_t, a_{t-1}, \hat{z}_t)$;
 $x_{t+1}, e_{t+1}, - \leftarrow \text{envs}[i].\text{step}(a_t)$;
 Store $(\hat{z}_t, z_t)$ in D_2 ;
 end
 end
 $\theta_\phi \leftarrow \theta_\phi - \lambda_{\theta_\phi} \nabla_{\theta_\phi} \frac{1}{TN_{\text{env}}} \sum \|\hat{z}_t - z_t\|^2$;
 Empty D_2 ;
end

with a very small k_0 set to 0.03, and then exponentially increase the these coefficients using a fixed curriculum: $k_{t+1} = k_t^{0.997}$, where t is the iteration number. The learning process is shown in Figure S2.

S1. ADDITIONAL REAL-WORLD ADAPTATION ANALYSIS

In addition to the oil-walking experiments in Figure 4 of the main paper, we also analyze the gait patterns and the torque profile for the mass adaptation case, shown in Figure S3. We throw a payload of 5kg on the back of the robot in the middle of a run and plot the torque profile of the knee, gait pattern, and the 2th and 7th components of the extrinsics vector $\hat{z}_t$ as shown in Figure S3. We observe that the additional payload disturbs the regular motion of the robot, after which it enters

Algorithm 2: Rapid Motor AdaptationDeployment

Process 1 operating at 100 Hz;

$t \leftarrow 0$;

while not fall do

$a_t \leftarrow \pi(x_t, a_{t-1}, \hat{z}_{\text{async}})$;

$x_{t+1} \leftarrow \text{env.step}(a_t)$;

$t \leftarrow t + 1$;

end

Process 2 operating at 10 Hz;

while not fall do

$\hat{z}_{\text{async}} \leftarrow \phi(x_{t-k:k}, a_{t-k-1:k-1})$;

end

S2. ADDITIONAL SIMULATION TESTINGS

In Figure S4, we further test RMA in extreme simulated environments and show its performance in three types of environment variations: the payloads added on the base of the A1 robot, the terrain elevation variation (z-scale used in the fractal terrain generator, details in Section IV Simulation Setup of the main paper), and the friction coefficient between the robot feet and the terrain. We show the superiority of RMA across all the cases in terms of Success Rate, TTF and Reward as defined in Section S1.

the adaptation phase and finally recovers from the disturbance. When the payload lands on the robot, it is noticeable that the plotted components of the extrinsics vector change in response to the slip. Post adaptation, we see that the torque stabilizes to a higher magnitude than before to account for the payload and the gait time period is roughly recovered.

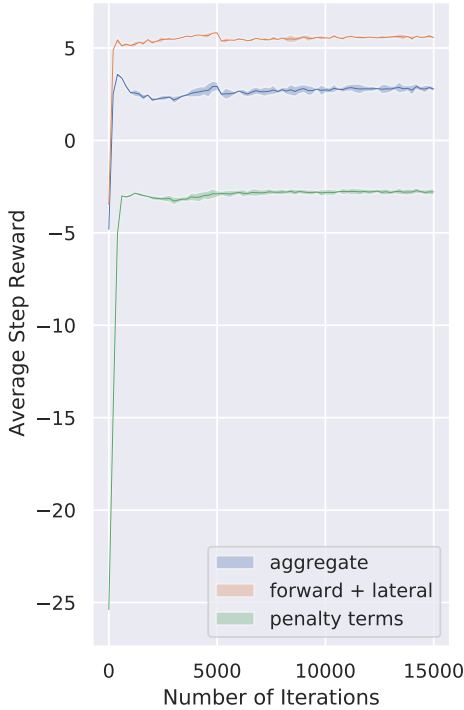


Fig. S2: We plot the average step reward during the total 15,000 training iterations. We show the converging trend of the reward aggregating all reward terms, forward + lateral reward, and sum of penalty terms. It also shows the necessity of applying a small multiplier to the penalty terms at the beginning of training; otherwise, the robot will only have negative experience initially and unable to learn to walk quickly.

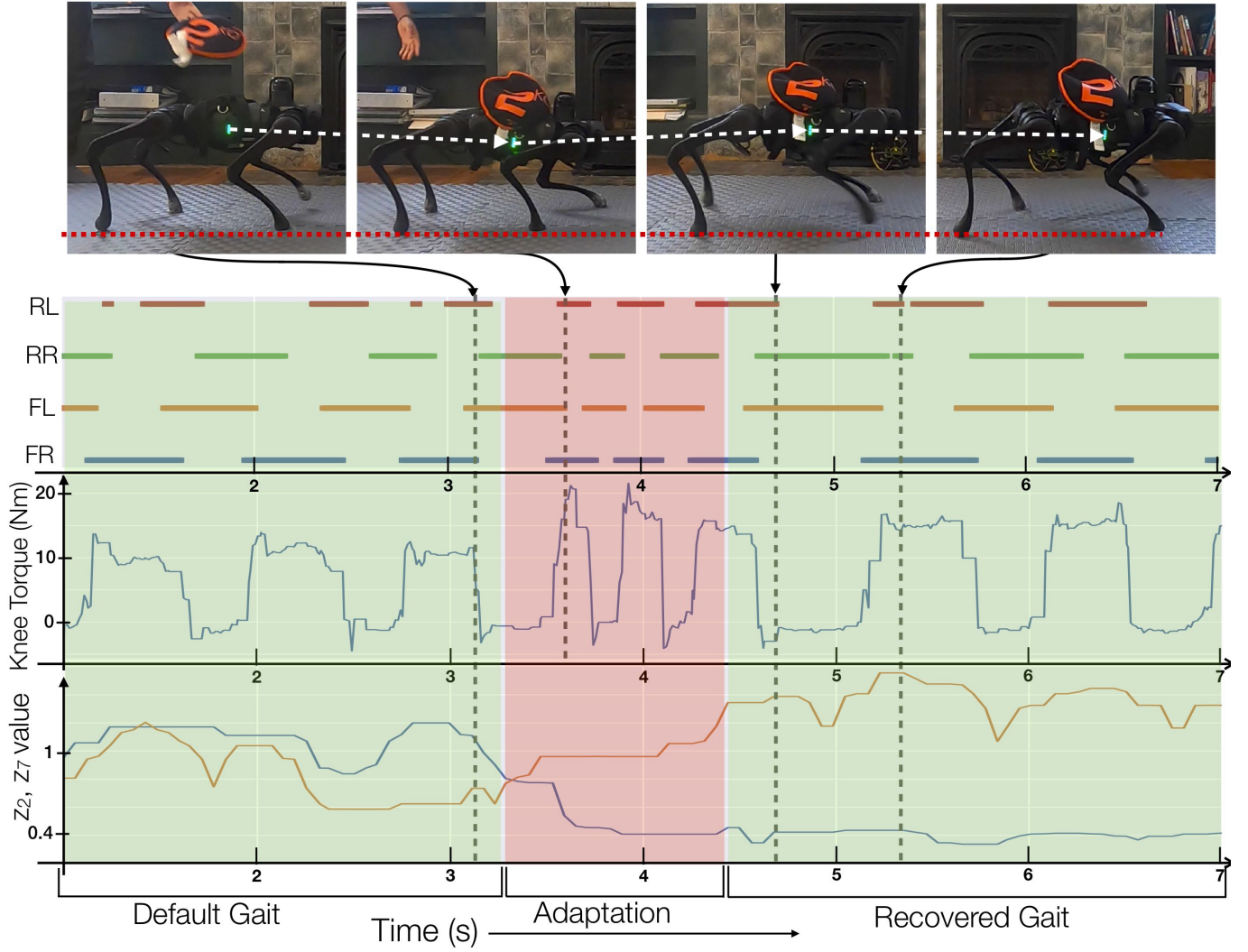


Fig. S3: We analyze the change in behavior of RMAas we throw a payload of 5kg on the back of the robot. As a note, we have flipped the images so that that movement appears from left to right which is why the label on the sandbag appears to be 2Kg. We plot the torque profile of the knee and the gait pattern. The bottom plot shows median filtered 2nd and 7th components of the extrinsics vector $\hat{z}$ predicted by the adaptation module. When the 5kg payload is thrown on the back of the robot, we see a dip in the center of mass of the robot, which the adaptation module subsequently recovers from. In the bottom plot, we see a jump in response in the plotted components of the estimated extrinsics vector, indicating that the additional payload has been detected by the adaptation module. Note that post adaptation, the recovered gait time period is roughly similar to the original, the torque magnitudes have increased and the extrinsics vector continues to capture the presence of the 5Kg payload on the back of the robot.

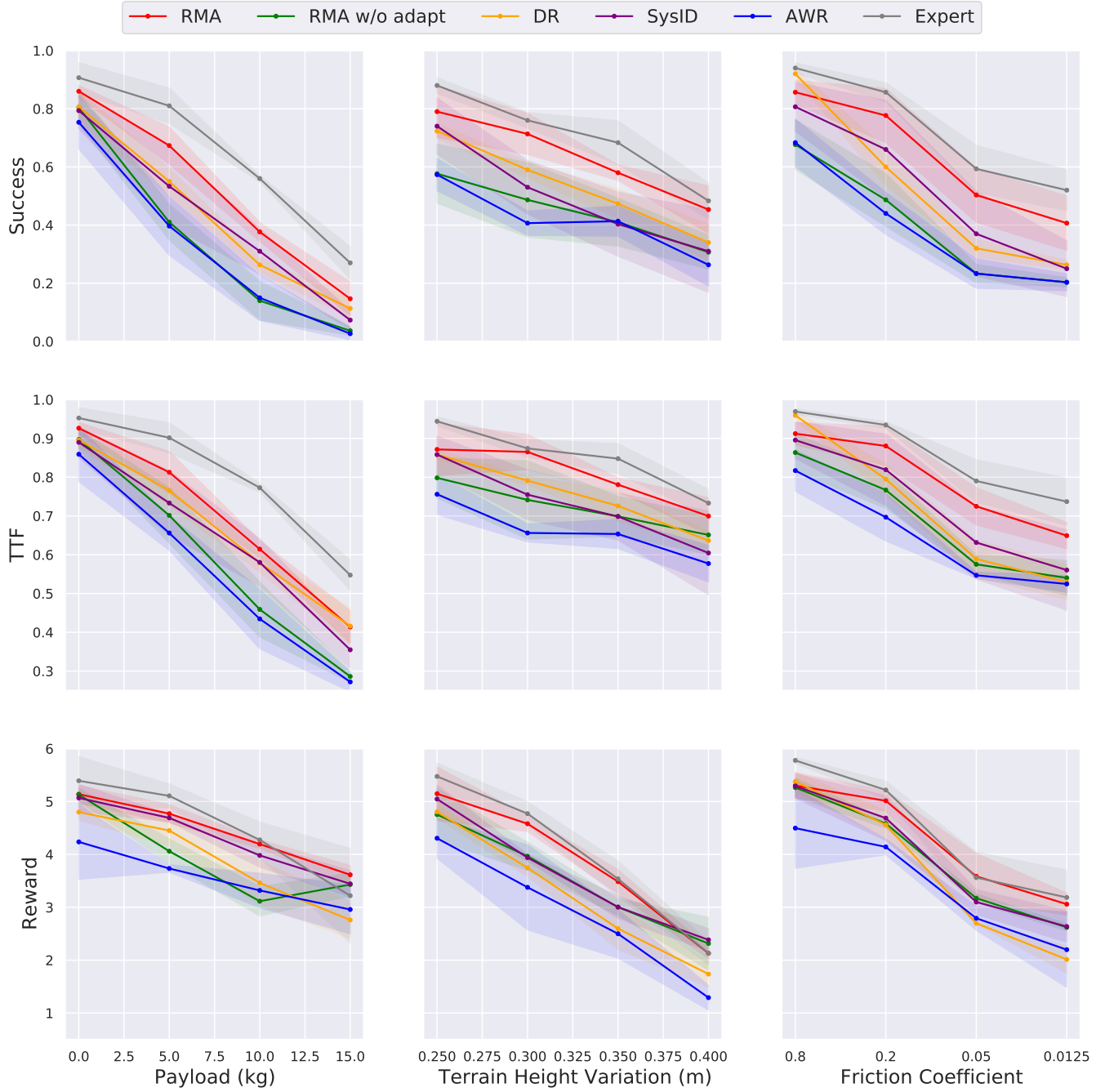


Fig. S4: Simulation Generalization Results: We further compare the generalization performance of our method to baseline methods in simulation. We pick three physics parameters that may vary to a large degree in the real world: the payload on robot, the terrain height variation, and the friction coefficient between the robot feet and the terrain. We set other environment parameters according to the training range in TABLE II of the main paper. Baselines and metrics are defined in Section V of the main paper and Section S1. We report the mean and standard deviation of the performance of 3 randomly initialized policies, which is characterized by the average of 100 testing trials in given settings. Despite no testing environment samples, RMA performs the best, the closest to Expert’s performance. For reference, A1 robot without additional payloads weighs 12 kg, and is 0.35 m tall. The static friction coefficient between rubber and concrete is 1.00.